NAME – SUBASH G
REG NO – 192572150

COURSE NAME – Operation System
COURSE CODE – CSA0408

Lab Observations

EXPERIMENT - 13

Construct a C program for implementation of the various memory allocation strategies.

PROGRAM:

```
#include <stdio.h>

void firstfit(int mp[], int p[], int m, int n)
{
    int allocated[20] = {0};

    for (int i = 0; i < m; i++)
    {
        int j;

        for (j = 0; j < n; j++)
        {
            if (!allocated[j] && mp[j] >= p[i])
            {
                printf("\nProcess of size %d fits in partition %d",
                       p[i], mp[j]);
                allocated[j] = 1;
                break;
            }
        }

        if (j == n)
            printf("\nProcess of size %d must wait", p[i]);
    }
}
```

```
void bestfit(int mp[], int p[], int m, int n)
{
    int allocated[20] = {0};

    for (int i = 0; i < m; i++)
    {
        int best = -1;

        for (int j = 0; j < n; j++)
        {
            if (!allocated[j] && mp[j] >= p[i])
            {
                if (best == -1 || mp[j] < mp[best])
                    best = j;
            }
        }

        if (best != -1)
        {
            printf("\nProcess of size %d fits in partition %d",
                   p[i], mp[best]);
            allocated[best] = 1;
        }
        else
        {
            printf("\nProcess of size %d must wait", p[i]);
        }
    }
}

void worstfit(int mp[], int p[], int m, int n)
{
    int allocated[20] = {0};

    for (int i = 0; i < m; i++)
    {
        int worst = -1;

        for (int j = 0; j < n; j++)
        {
            if (!allocated[j] && mp[j] >= p[i])
            {
                if (worst == -1 || mp[j] > mp[worst])
```

```
                    worst = j;
            }
        }

        if (worst != -1)
        {
            printf("\nProcess of size %d fits in partition %d",
                   p[i], mp[worst]);
            allocated[worst] = 1;
        }
        else
        {
            printf("\nProcess of size %d must wait", p[i]);
        }
    }
}

int main()
{
    int m, n, mp[20], p[20], ch;

    printf("Number of memory partitions: ");
    scanf("%d", &n);

    printf("Number of processes: ");
    scanf("%d", &m);

    printf("Enter the memory partitions:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &mp[i]);

    printf("Enter process sizes:\n");
    for (int i = 0; i < m; i++)
        scanf("%d", &p[i]);

    printf("\n1. First Fit\n2. Best Fit\n3. Worst Fit");
    printf("\nEnter your choice: ");
    scanf("%d", &ch);

    switch (ch)
    {
        case 1:
            firstfit(mp, p, m, n);
```

```
            break;

        case 2:
            bestfit(mp, p, m, n);
            break;

        case 3:
            worstfit(mp, p, m, n);
            break;

        default:
            printf("Invalid choice");
    }

    return 0;
}
```

OUTPUT:

